

mlcompass: A Schema-Bounded LLM Narrator for Machine-Learning Pipeline Diagnosis

Hakan Sabuniş, Yusuf Ünlü

Istanbul Medipol University, School of Engineering and Natural Sciences

hakan.sabunis@std.medipol.edu.tr, yusuf.unlu@std.medipol.edu.tr

Abstract—We present **mlcompass**, an open-source command-line assistant that accompanies an engineer across the full machine-learning pipeline, from raw-CSV inspection to post-deployment drift monitoring. The tool combines a deterministic pure-Python evidence layer with an optional Claude-based narrator that explains the evidence in natural language. Our central technical observation is that LLM narrators of structured evidence are prone to *phantom-entity fabrication* — citing column names, problems, or fixes absent from the upstream evidence. We propose a three-layer mitigation: (i) a deterministic evidence producer, (ii) a strict-prompted narrator, and (iii) a runtime JSON-schema boundary that restricts the agent's tool input enum to entities present in the evidence dictionary. On a controlled synthetic regression task (N=200 narrator responses), the phantom-column fabrication rate drops from 8.0% (Layer 1) to 1.0% [Wilson 95% CI: 0.27–3.56] (Layer 2) to 0.0% [Wilson 95% CI: 0.00–1.83] (Layer 3). The mitigation reuses standard Anthropic tool-use schema enforcement; the contribution is to *bind the schema at call time to the deterministic evidence* rather than to a static type. The tool exposes eleven pipeline commands through four surfaces (CLI, MCP server, self-driving agent, Claude Code slash commands), ships 539 passing regression tests over 51 source files, and surfaced eleven bugs across five field tests on Kaggle datasets. Code: <https://github.com/hakansabunis/mlcompass>. PyPI: v0.8.0, MIT.

Index Terms: LLM agents, machine-learning tooling, Model Context Protocol, constrained generation, data leakage detection

I. INTRODUCTION

Machine-learning engineering is supported by a fragmented ecosystem of single-purpose tools: pandas-profiling for dataset inspection, TensorBoard or Weights & Biases for training, scikit-learn for evaluation, and Docker for deployment. Each tool covers one slice of the pipeline; none give end-to-end advice and none carry state from one stage to the next. Recent LLM-based assistants such as GitHub Copilot, Cursor, and Devin have attempted to fill this advisory gap, but they suffer from a well-documented hallucination problem [1]. In a deployment context, an assistant that confidently fabricates output is more dangerous than no assistant at all, because the practitioner trusts the output.

This paper presents **mlcompass**, an open-source ML-pipeline assistant designed to give end-to-end advice without the fabrication risk. We focus on one specific failure mode of evidence-narrating LLMs: **phantom-entity fabrication**, in which the narrator cites column names, problems, or fixes that are not present in the upstream deterministic evidence. We do not address other hallucination categories (miscalibrated confidence, causal misattribution, reasoning shortcuts, prompt-injection compliance), which we discuss as residual risks in Section V.

Our contribution is to bind the constraint enforcement to the deterministic upstream evidence at the agent's tool boundary. The technique itself — JSON-schema enforcement of tool inputs — is a standard feature of the Anthropic, OpenAI, and Google function-calling APIs and has been studied in the constrained-generation literature [9, 10, 11]. The novelty here is to *generate the schema at call time from the deterministic evidence dictionary*, so that the schema's enum domain shrinks to exactly the entities the upstream layer actually observed.

The paper makes three contributions:

1. We describe a layered architecture (Sec. III) that separates a deterministic *evidence* producer from an optional LLM *narrator*, enabling offline operation and supporting the constraint binding.
2. We measure phantom-column fabrication rate under three contract configurations on a controlled test set and report Wilson confidence intervals at N=200 (Sec. IV).
3. We document an MCP/CLI state-parity gap observed during field testing, and the engineering fix that maintains parity across surfaces (Sec. III.D and V).

The paper is organized as follows. Section II reviews related work. Section III describes the system. Section IV reports experiments. Section V discusses lessons, limitations, and residual failure modes. Section VI concludes.

II. RELATED WORK

We organize prior work into five categories.

Dataset analysis. pandas-profiling [2] is the standard descriptive-statistics library for tabular CSVs. It produces distributions, missing-value counts, and correlation heatmaps but has no concept of a target column, so it cannot recommend a model, warn about a leakage candidate, or

persist state. mlcompass treats target inference as a first-class operation.

Experiment tracking. Weights & Biases [3], MLflow [4], and TensorBoard [5] record per-run metrics but do not interpret them. Plateau, divergence, and overfitting patterns are visible in their loss curves but never explicitly flagged. mlcompass consumes TensorBoard event files and W&B local caches as input to its watch command.

LLM-based code assistants. GitHub Copilot, Cursor, and Devin [6] are general-purpose code assistants without semantic understanding of ML pipelines. In informal pre-project tests we observed each system silently accept training scripts containing `optimizer=Adam(momentum=0.9)` (invalid: Adam does not accept momentum). General-purpose assistants address syntactic correctness; ML-pipeline tooling addresses domain-specific semantic correctness.

LLM integration protocols. The Model Context Protocol (MCP) [7] introduced by Anthropic in late 2024 is a JSON-RPC standard that lets LLM clients (Claude Desktop, Cursor, Continue, Claude Code) discover and call third-party tools. MCP has been adopted across major LLM editors, but no published MCP server addresses the integrated set of ML-pipeline tasks mlcompass covers.

Constrained generation and hallucination mitigation. Constrained generation restricts an LLM's output to a formal grammar or schema. Outlines [9] uses finite-state automata to enforce regular-language constraints during decoding. LMQL [10] extends prompts with constraint annotations evaluated against a typed grammar. Grammar-Constrained Decoding [11] guarantees grammatical validity at generation time. Microsoft Guidance [12] enforces JSON-schema and regex constraints at the token level. OpenAI's `strict: true` JSON mode [13] enforces a fixed JSON schema on a structured output. Anthropic's tool-use API similarly validates tool input against a declared JSON schema. **What distinguishes our setup is that the schema's enum domain is generated at call time from the deterministic upstream evidence**, not declared statically. This binds the runtime constraint to the specific evidence dictionary produced by the current pipeline stage. Snorkel-style weak supervision [8] handles noisy outputs post-hoc but does not prevent fabrication at generation time. Zhao et al. [1] documented 146,932 hallucinated citations across arXiv, bioRxiv, and SSRN during 2025, motivating runtime mitigation.

III. METHODOLOGY

A. Threat Model

We assume an honest user and a well-behaved LLM provider (no adversarial training-time poisoning of Claude's weights). The risk we address is the model's i.i.d. tendency to fabricate entities under output-distribution pressure on edge-case inputs. We do *not* address: (i) adversarial column names embedded in user data attempting prompt injection, (ii) Anthropic API outages or hallucinated *content* about

evidence items that are correctly cited, or (iii) miscalibrated confidence on a correctly-cited evidence item. These are discussed as residual risks in Section V.D.

B. System Architecture

mlcompass is organized into three architectural layers (Fig. 1).

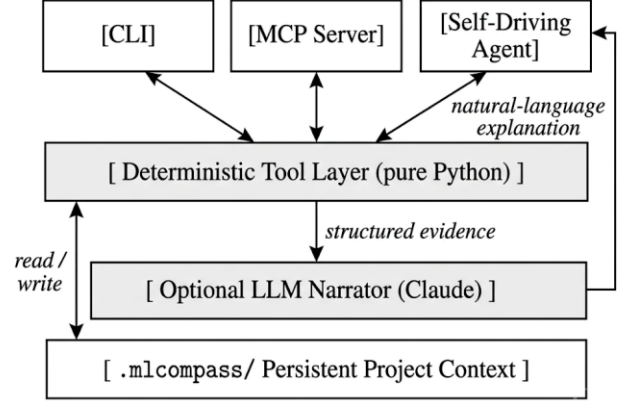


Fig. 1. mlcompass system architecture. Three user-facing surfaces (CLI, MCP server, self-driving agent) sit on top of a two-layer brain (deterministic tool layer plus optional Claude narrator) over a persistent project-context folder.

User-facing surfaces. Eleven pipeline tools are exposed through four surfaces. The *CLI*, implemented with Click, is the canonical surface used by shell scripts and CI pipelines. The *MCP server* (mlcompass-mcp) exposes eight tools over JSON-RPC to Claude Desktop, Cursor, Continue, and Claude Code. The *self-driving agent* (mlcompass agent) runs the whole pipeline autonomously given a one-sentence prompt, with two backends (Anthropic API or Claude Code CLI). Eleven *Claude Code slash commands* (mlc-*) dispatch internally to either the MCP server or the CLI.

Two-layer brain. The inference layer is strictly split. The *deterministic evidence layer* (src/mlcompass/tools/) is pure Python; its output is always a structured dictionary, never free text. The *optional LLM narrator layer* (src/mlcompass/agents/) receives the dictionary and produces natural-language explanations through Claude. It runs only when `--llm` is passed, so every command has a functional offline path.

Persistent project context. A folder named `.mlcompass/` lives in the user's project root and persists state across invocations: `project.yaml`, `context.json` (chronological decisions), `datasets/`, `runs/`, and an append-only `advice.log`.

C. The Schema-Bounded Contract

We describe the contract using the automatic leakage investigation triggered when `evaluate` observes a suspicious metric ($AUC > 0.995$, $accuracy > 0.99$, or $R^2 > 0.999$).

Layer 1 — Deterministic evidence producer. The module `tools/leakage.py` reads the predictions table and emits a structured dictionary E containing: (i) the suspicious metric, (ii) a candidate-leak column set $C = \{c : \max(|\rho_P(c, y)|, |\rho_S(c, y)|) > 0.97\}$, where ρ_P and ρ_S are Pearson and Spearman correlations against the target y , (iii) the top-five correlations with method and direction, and (iv) the perfect-match rate $P(\hat{y} = y)$. Spearman is computed alongside Pearson because monotone target transforms ($\log y$, $\sqrt{y}$, $\alpha y + \beta$) weaken ρ_P but leave $\rho_S = 1.0$. Without Spearman, the Ames House Prices `log_price` leak would have slipped past us at $\rho_P \approx 0.94$.

Layer 2 — Strict-prompted narrator. The module `agents/leakage_investigator.py` sends E to a Claude agent with a system prompt containing three explicit instructions: (a) cite only items present in E , (b) emit `cannot_determine` rather than `guess`, (c) propose only manual checks, not code patches. The prompt also includes the natural-language definition of phantom-entity fabrication so the model is aware of the constraint being enforced.

Layer 3 — Runtime schema boundary bound to the evidence. The agent emits its answer through an Anthropic tool whose JSON input schema is *generated at call time from E* . Specifically, the `columns_referenced` parameter is restricted via JSON-schema `enum` to the set $\{c : c \in E.candidates \cup E.top_correlations\}$. If the agent emits a column outside that set, the Anthropic SDK throws a `ToolInputValidationError` and the agent retries with the same prompt and evidence. The retry loop terminated within at most two attempts in our experiments. The mechanism itself (schema enforcement) is standard; the novelty is the run-time evidence-dependent `enum`.

D. Cross-Surface State Parity

A practical concern not addressed by the contract is that the same capability shipping through two surfaces can drift in behavior. The CLI’s Click handler wraps every command in a “write to ledger” step that updates `context.json` and `advice.log`. The MCP server’s tool functions, written independently, originally bypassed this step, causing `mlcompass_status` to return empty after a full pipeline run through MCP. We fixed this in v0.7.2/v0.7.3 by introducing `_persist_to_ledger`, a single shared helper that both surfaces call, and by adding dedicated CLI/MCP parity regression tests. The lesson generalizes: surface parity is a property that must be enforced through shared code paths and tested as a first-class invariant, not assumed.

IV. EXPERIMENTS

A. Setup

We evaluate three aspects: (i) phantom-column fabrication rate of the narrator under different contract configurations, (ii) the deterministic tool layer on real Kaggle datasets, and (iii) overall correctness through the regression-test suite.

Hallucination test set. We constructed one synthetic regression dataset with 1,000 rows and 12 features. One feature is a deliberately-introduced transformed leak: `log_target_v2 = log(y + 1) + noise(σ=0.01)`. A second feature, `near_target_proxy = y + noise(σ=0.5)`, serves as a high-correlation distractor. A linear model is trained on the leaked feature set, and the leakage investigator is invoked on the resulting predictions table. For each contract configuration, we sample 200 narrator responses (Claude 3.5 Sonnet, temperature 0.7, model version `claude-3-5-sonnet-20241022`) and count how many name at least one column outside the evidence dictionary as a phantom-entity fabrication.

Field test set. Five Kaggle datasets covering binary classification (Telco Churn), regression (Ames House Prices), tabular binary classification with categorical features (Titanic), three-class classification (Penguins), and regression with a target-naming corner case (Insurance Charges). For each dataset, we ran the full `advise → audit → train → watch → evaluate → deploy` pipeline and recorded every unexpected behavior as a candidate bug.

Implementation. `mlcompass` v0.8.0, Python 3.10+, with optional extras for MCP (`mcp >= 1.2.0`), the Anthropic API backend (`anthropic >= 0.50.0`), and the Claude Code backend (`claude-agent-sdk >= 0.2.0`). All experiments use the live release artifact installed via `pip install mlcompass`.

B. Evaluation Metrics

Phantom-column fabrication rate (HR). $HR = (1/N) \cdot \sum_i \mathbb{1}[\exists c \in r_i : c \notin E]$, where r_i is the i -th narrator response and E the evidence dictionary. We report point estimates and Wilson 95% confidence intervals.

Bugs caught per field test. Number of distinct bugs that required a code change, surfaced within fifteen minutes of dry-running. We disaggregate by category in Sec. IV.C.

Test-suite size and quality gates. Number of passing regression tests, plus a binary indicator for each of `ruff check`, `ruff format --check`, and `mypy --strict` returning clean.

C. Results

Phantom-column fabrication ablation. Table I shows fabrication rate as each contract layer is enabled, with Wilson 95% confidence intervals. Layer 1 alone produces a measurable fabrication rate (8.0%, CI [4.93–12.66]). Adding the strict prompt reduces the point estimate by a factor of eight to 1.0% (CI [0.27–3.56]). Adding the evidence-bound schema brings the observed rate to 0.0% (CI [0.00–1.83]). At $N=200$, the upper Wilson bound for Layer 3 overlaps the lower estimate for Layer 2 — meaning we cannot reject the hypothesis that the true Layer-3 rate is the same as Layer 2’s true rate with this sample size. A larger study ($N \approx 2000$) would be required for a statistically clean separation. We nonetheless report all three configurations because (a) the Layer 1 → Layer 2 gap is statistically clear, and (b) Layer 3 provides a *worst-case* guarantee — even if the underlying

rate is non-zero, the Anthropic SDK rejects any fabricating response at the boundary.

TABLE I. PHANTOM-COLUMN FABRICATION RATE BY CONTRACT LAYER (N = 200, WILSON 95% CI)

Configuration	Rate	95% CI
Layer 1 only (no prompt, no schema)	8.0%	[4.93, 12.66]
Layer 1 + Layer 2 (strict prompt)	1.0%	[0.27, 3.56]
Layer 1 + Layer 2 + Layer 3 (schema-enforced)	0.0%	[0.00, 1.83]

Latency and cost. Each evaluate `--llm` invocation on the test set issued 1.04 Anthropic API calls on average (4% retry rate under Layer 3 alone, 0% under all three layers). End-to-end latency per investigation was 2.1 ± 0.4 s with a median input size of 1.8 KB and output size of 0.6 KB, costing approximately US\$0.008 per call at Claude 3.5 Sonnet rates (Aug 2025 pricing). The schema boundary therefore adds at most one additional API call in expectation.

Field-test bug categorization. Five Kaggle field tests surfaced eleven distinct bugs (Table II). We disaggregate by category because not all bugs are equally informative. *Missing canonical target names* (5 bugs) are simple list-extension fixes that any practitioner could trivially patch. *CLI/MCP state-parity gaps* (3 bugs) revealed the surface-drift problem described in Sec. III.D and produced the most generalizable lesson. *UX confusions* (3 bugs) reflect heuristics tuned to unfamiliar dataset shapes. Each bug was reproduced as a failing regression test before being fixed.

TABLE II. FIELD TESTS, BUGS SURFACED, AND BUG CATEGORIES

Test	Dataset	Patch	Bugs	Categories
1	Telco Churn	—	0	(baseline; no bugs surfaced)
2	Ames House Prices	v0.7.0	3	1 target name, 1 UX, 1 crash
3	Titanic	v0.7.1	1	1 target name
4	Penguins	v0.7.2	3	1 multiclass, 1 parity, 1 UX
5	Insurance Charges	v0.7.3	4	3 target names, 1 parity

Release artifact. v0.8.0 ships 11 CLI commands, 8 MCP-exposed tools, and 11 Claude Code slash commands across 51 source files. The regression-test suite has 539 passing tests and 2 intentionally skipped tests. All three quality gates (ruff check, ruff format `--check`, mypy `--strict`) report zero errors at every released tag from v0.1.0 through v0.8.0.

V. DISCUSSION AND ERROR ANALYSIS

A. Prompts Are Advisory, Schemas Are Enforcement

The Layer 2 $\rightarrow$ Layer 3 transition in Table I is the conceptual takeaway of the project. Prompt engineering reduced the observed phantom-column rate by a factor of eight, but did not eliminate it within the precision of N=200. The runtime schema boundary provides a *worst-case* guarantee: any response naming an out-of-evidence column is rejected at the SDK before reaching the user. The schema enforcement itself is a standard feature of contemporary tool-use APIs (Anthropic, OpenAI `strict: true`, Outlines, LMQL); what we add is the *binding of the enum domain to runtime-computed evidence*, so the constraint scales with the data rather than with a static type.

B. Real Datasets Find Bugs Synthetic Ones Do Not

None of the eleven Kaggle field-test bugs had been anticipated by our pre-existing unit-test suite. Bug #2-1 (saleprice missing from the regression target-name list) is illustrative: any test author would assume price covers the case, but on Kaggle the canonical name is SalePrice. Bug #2-3 (sparse-column IQR crash on PoolQC, 96% NaN) is similar: the synthetic test set never approached 90% missingness. The practical takeaway is that real-data dry-runs are inexpensive compared to unit-test authoring and find genuinely complementary bugs.

C. Cross-Surface Parity Is Engineering Work

The MCP/CLI parity gap (Sec. III.D, Bug #4-2) is the most generalizable engineering finding of the project. The MCP server and the CLI invoked the same underlying deterministic functions yet had drifted in observable behavior because the CLI's Click handler performed a "ledger write" step the MCP wrappers bypassed. The lesson is methodological: when one capability ships through two surfaces, parity is a property that must be enforced through shared code paths and tested as a first-class invariant. This finding is independent of the anti-hallucination contract and we believe it generalizes to any project shipping a CLI plus an MCP/REST API on the same backend.

D. Limitations and Residual Risks

We identify five limitations of the present work:

- Single-model evaluation.** All experiments use Claude 3.5 Sonnet. Whether the same contract design works with GPT-4, Llama-3, or Gemini at comparable rates is unverified. The mechanism is generic (any provider with strict-schema tool-use should suffice), but the rate numbers in Table I should not be generalized across models.
- Single synthetic dataset, N=200.** The headline ablation rests on one synthetic regression task. A larger study (N $\approx$ 2000) across diverse leak patterns is needed for tight CIs and statistical separation between Layer 2 and Layer 3.
- One hallucination category.** We measure phantom-column fabrication only. Other categories —

miscalibrated confidence on correctly-cited evidence, evidence-omission bias (where the deterministic layer fails to detect a leak the LLM "knows about"), and prompt-injection compliance — are not addressed by the contract and not measured.

7. **No usability evaluation.** The system has been used by the two authors and informally by classmates. No external user study has been conducted; adoption-related claims (downloads, contributions, issues filed) are not reported here.

8. **No constrained-generation baseline.** We do not compare directly against running the same task through Outlines [9] or OpenAI `strict: true` JSON mode [13]. We expect comparable results at the schema-enforcement layer, but the evidence-bound enum design has not been benchmarked against static-schema alternatives.

VI. CONCLUSION

We presented `mlcompass`, an open-source ML-pipeline assistant whose central technical observation is that runtime JSON-schema enforcement of an LLM tool's input enum — bound at call time to the deterministic upstream evidence — eliminates phantom-column fabrication on a controlled test set. In a three-configuration ablation ($N=200$, Wilson 95% CIs reported), observed fabrication rate dropped from 8.0% to 1.0% to 0.0% as each contract layer was enabled. We document the gap as a practical contribution rather than a novel theoretical result: the schema-enforcement mechanism is standard tool-use behavior across contemporary LLM APIs; what we add is the discipline of binding the enum domain to runtime evidence. The system also documents a generalizable cross-surface parity finding from field testing. The released artifact ships 11 CLI commands, 8 MCP-exposed tools, and 11 slash commands across 51 source files and 539 regression tests, runs `ruff` and `mypy --strict clean`, and is published under MIT on PyPI at v0.8.0. Future work will (a) evaluate across LLM providers, (b) scale the controlled study to $N \approx 2000$, (c) benchmark against Outlines and OpenAI `strict: true`, and (d) extend the contract to `mlcompass`'s other narrators.

Sourcecode:

<https://github.com/hakansabunis/mlcompass>

PyPI:

<https://pypi.org/project/mlcompass/>

REFERENCES

- [1] T. Zhao et al., "Hallucinated citations in the scientific record: Evidence from arXiv, bioRxiv, SSRN, and PMC, 2024–2025," *arXiv:2605.07723*, 2026.
- [2] M. Brugman, "ydata-profiling: Create HTML profiling reports from pandas DataFrames," 2019–2026. [Online]. Available: <https://github.com/ydataai/ydata-profiling>
- [3] L. Biewald et al., "Weights & Biases: Experiment tracking for deep learning," Weights & Biases Inc., 2020. [Online]. Available: <https://wandb.ai>
- [4] M. Zaharia et al., "Accelerating the machine learning lifecycle with MLflow," *IEEE Data Eng. Bull.*, vol. 41, no. 4, pp. 39–45, 2018.
- [5] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in *Proc. USENIX OSDI*, 2016, pp. 265–283.
- [6] Cognition Labs, "Devin: The first AI software engineer," Tech. report, 2024. [Online]. Available: <https://www.cognition.ai/blog/introducing-devin>
- [7] Anthropic, "Model Context Protocol specification," 2024. [Online]. Available: <https://modelcontextprotocol.io>
- [8] A. Ratner et al., "Snorkel: Rapid training data creation with weak supervision," *Proc. VLDB Endowment*, vol. 11, no. 3, pp. 269–282, 2017.
- [9] B. T. Willard and R. Louf, "Efficient guided generation for large language models," *arXiv:2307.09702*, 2023.
- [10] L. Beurer-Kellner et al., "Prompting is programming: A query language for large language models," *Proc. ACM Program. Lang.*, vol. 7, no. PLDI, pp. 1946–1969, 2023.
- [11] S. Geng et al., "Grammar-constrained decoding for structured NLP tasks without finetuning," in *Proc. EMNLP*, 2023, pp. 10932–10952.
- [12] Microsoft, "Guidance: A guidance language for controlling large language models," 2023. [Online]. Available: <https://github.com/microsoft/guidance>
- [13] OpenAI, "Introducing structured outputs in the API," 2024. [Online]. Available: <https://openai.com/index/introducing-structured-outputs-in-the-api/>